

四叶草平台五项目整合

Docker 部署与运行规范

项目	内容
版本	v1.1（基于确认版约束）
日期	2026-05-18
适用范围	Portal、合同审查、标书生成、RAG 问答、企业竞对分析
关键约束	PostgreSQL 18；第一阶段保留 iframe；暂用现有认证；数据短期共享；配置多环境化

0. 已确认约束

确认项	结论
PostgreSQL 版本	开发、测试、生产均统一使用 PostgreSQL 18。
认证体系	第一阶段复用现有 Portal 登录与 session 机制，平台稳定后再升级认证体系。
多租户	当前不做复杂多租户，但表结构预留 tenant_id / organization_id 等扩展口子。
标书生成服务边界	短期保留标书生成现有独立逻辑，中期合并为 bid_generator 模块。
任务队列	第一阶段不引入 Celery/RQ，后续按长任务压力再评估。
iframe 策略	第一阶段保留 iframe，第二阶段统一后端和数据库，第三阶段逐步去 iframe。
文件存储	暂时使用本地目录或 Docker volume，后续可平滑切换 MinIO。
权限策略	默认普通用户可使用五个模块，管理员可配置某用户不能使用某模块；所有数据短期共享。
数据隔离	暂时不做隔离，但业务表和查询接口预留 user_id、tenant_id、visibility、scope 等口子。
多环境配置	配置体系支持 dev/test/prod 多环境，Dify workflow key 按环境隔离。

1. 部署原则

Docker 部署不依赖动态端口。生产或准生产环境应使用固定容器内部端口，通过 Docker 网络互联，只对外暴露 web 和 api。PostgreSQL 18 使用独立容器和持久化 volume。

服务	端口	说明
web	5200	统一前端入口。
api	5210	统一后端 API。
postgres	5432，仅内部	PostgreSQL 18，不建议直接暴露公网。
legacy services	内部端口	阶段一保留时可作为内部服务，不直接面向用户。

2. docker-compose 示例

```
services:
  postgres:
    image: postgres:18
    environment:
      POSTGRES_DB: clover_platform
      POSTGRES_USER: clover
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks: [clover]

  api:
```

```

build:
  context: ..
  dockerfile: docker/Dockerfile.api
environment:
  APP_ENV: ${APP_ENV:-dev}
  DATABASE_URL:
postgresql+psycopg://clover:${POSTGRES_PASSWORD}@postgres:5432/clover_platform
  DIFY_BASE_URL: ${DIFY_BASE_URL}
  depends_on: [postgres]
  ports:
    - "5210:5210"
  volumes:
    - app_storage:/app/storage
  networks: [clover]

web:
  build:
    context: ..
    dockerfile: docker/Dockerfile.web
  depends_on: [api]
  ports:
    - "5200:5200"
  networks: [clover]

volumes:
  postgres_data:
  app_storage:

networks:
  clover:

```

3. 环境变量规范

变量	说明
APP_ENV	dev/test/prod。
DATABASE_URL	统一数据库连接。
POSTGRES_PASSWORD	PostgreSQL 密码。
SECRET_KEY	session/token 加密密钥。
DIFY_BASE_URL	Dify API 地址。
DIFY_API_KEY	Dify API key。
DIFY_WORKFLOW_*	各环境 workflow key。
STORAGE_BACKEND	local，未来可为 minio。

4. 数据卷规范

- postgres_data：PostgreSQL 数据持久化。
- app_storage：合同、标书、知识库、导出文件等本地文件。
- logs：可选，保存应用日志。
- 后续切换 MinIO 时，core.files.storage_backend 从 local 切换为 minio，业务代码不直接感知。

5. 启动和初始化

```
# 1. 准备环境变量
cp .env.example .env

# 2. 启动基础服务
docker compose -f docker/docker-compose.yml up -d postgres

# 3. 初始化数据库 schema 和表
python scripts/migrate.py upgrade head

# 4. 启动平台
docker compose -f docker/docker-compose.yml up -d --build
```

6. 健康检查和回滚

- web 健康检查: GET /。
- api 健康检查: GET /api/v1/core/health。
- 模块健康检查: GET /api/v1/{module}/health。
- 回滚优先回滚镜像版本, 不直接修改数据库。
- 数据库结构变更必须保留 migration 文件, 开发阶段可 reset, 生产阶段必须备份。